

INTERNAL — CLIENT DEMO

dockwatch Demo

Runbook

~15 minute walkthrough covering local Docker monitoring, Portainer integration, and the safety/audit features that differentiate dockwatch from "just check for a new tag" tools.

DURATION	SECTIONS	GENERATED
~15 min	6	2026-08-12



✓ Before the call

Confirm the environment is up:

```
docker ps --format "{{.Names}}\t{{.Status}}"
```

Expect to see (adjust names to your actual demo host):

Container	Role in demo
<code>dockwatch</code>	the app itself
<code>portainer</code>	Portainer instance, shows the "managed host" story
<code>dwtest-stack-web</code>	Portainer-managed compose stack — update/restart/delete via Portainer
<code>plausible</code> , <code>plausible-db</code> , <code>plausible-events-db</code>	multi-service local compose stack — realistic "customer's actual app"
<code>jackett</code>	local container, real outdated version available — guaranteed "OUTDATED" row
<code>uptime-kuma</code>	local container, second up-to-date/outdated example — good disposable target for the delete demo in section 5

Log into dockwatch at `http://<host>:10801` before the call starts. Have Portainer open in a second tab (`http://<host>:9000` or `:9443`) to jump into if a question calls for "what does Portainer see."

1 The core problem

30 sec — no screen share needed

"You have containers running versions from six months ago and no way to know without manually checking every image tag. dockwatch checks your registries on a schedule and tells you what's outdated, what changed, and lets you act on it — without you needing to SSH in or remember every stack."

2 Dashboard tour

2 min

- Point out the status column: **Up-to-date / Outdated / Pinned / Local / Unknown**.
- Click **Check now** — show it hitting real registries live.
- Point at `jackett` 's row: OUTDATED, shows current tag vs. available tag, bump-type badge (MAJOR/MINOR/PATCH color-coded).
- Point at `plausible` : correctly shows up-to-date, digest-based comparison (not just tag string matching) — explain floating-tag containers (`latest`) are only trusted when digest

comparison is available, otherwise flagged.

3 Local update flow

2 min

- Click **Update** on `jackett`.
- Narrate: dockwatch rewrites the compose file's pinned tag, runs `docker compose pull jackett` + `up -d jackett`, and the container restarts on the new image. Only `jackett` is touched — sibling services in the same compose file stay running untouched.
- Open **History** (clock icon) on the row afterward — show the audit log: who did it, when, old tag → new tag, success/failure.
- Mention: rollback is one click from the same panel if an update goes bad.

4 Portainer integration

4 min — the differentiator

- Switch dashboard source to **Portainer** in the toolbar.
- Show `dwtest-stack-web` appears with a Portainer badge and environment name.
- Explain: this container might be on a completely different host — dockwatch never touched its Docker socket directly, everything routes through Portainer's own API.
- Click **Restart** on it — show it actually restarts (new container start time if asked to prove it, `docker inspect dwtest-stack-web --format '{{.State.StartedAt}}'`).
- If it's showing OUTDATED: click **Update** — same tag-rewrite-and-redeploy story, but done through Portainer's stack API instead of a local compose file. Only the target service's image line is rewritten; Portainer diffs the stack file and recreates only the changed service. This is the harder problem most competitors don't solve — updating containers you don't have filesystem/socket access to.

5 Safety and admin controls

3 min

- Open **Users/Roles** (admin only) — show permission model: `view_containers`, `update_containers`, `delete_containers`, `scan_containers`, `manage_settings`, `manage_users`.
- Point out: delete requires its own explicit permission, separate from update — a team can let people update containers without letting them delete anything.
- Demonstrate a delete: pick a disposable test container, click **Delete**, show the confirmation dialog (no silent destructive actions), show it logged in history same as any other action.
- Mention Trivy vulnerability scanning if the client's compliance-conscious: per-container CVE scan, severity-filtered findings list.

6 Q&A anchors

Likely questions and the honest answer:

Question	Answer
"Does it work across multiple Docker hosts?"	Yes, via Portainer environments — one dockwatch instance, many hosts.
"Can it update anything, or just compose stacks?"	Local: compose-managed and plain <code>docker run</code> containers both supported. Portainer: compose-managed (stack-deployed) containers only — a bare Portainer container can be restarted/deleted but not pull-updated yet.
"What if an update breaks something?"	Rollback button reverts to the last known-good tag for compose-managed containers; plain-mode local updates auto-rollback on failure during the update itself.
"Is this agent-based?"	No agent to install on target hosts — local mode uses the Docker socket directly, remote hosts go through Portainer's existing API.
"What's NOT supported yet?"	Full recreate (network/volume changes, not just image tag) via Portainer; Kubernetes environments.

AFTER THE CALL

Nothing to reset — the demo containers are safe to leave running and demo again. If a delete was demonstrated on a disposable container, recreate it before the next demo:

```
docker run -d --name <name> nginx:alpine
```